UNIVERSITÉ MUNDIAPOLIS

3A DS AI

RAPPORT DE TRAVAUX PRATIQUES

TP 4

API Iris et compteur Redis

Cloud Computing et Déploiement IA

Réalisé par

Aya Kmaiti

Année universitaire 2026–2027

Table des matières

Objectif du TP **1**

1 Préparer le projet existant **1**

2 Ajouter Redis à l'API **2**

3 Configurer Docker Compose **3**

4 Démarrer et observer les services **4**

5 Vérifier le compteur initial **5**

6 Envoyer cinq prédictions **6**

7 Tester la persistance du compteur **7**

Objectif du TP

Nous reprenons l'API de classification Iris du TP précédent pour lui ajouter une mémoire avec Redis. L'API réalise les prédictions, tandis que Redis conserve leur nombre. Les tests vérifient ensuite si ce compteur survit à un arrêt et à une recréation des conteneurs.

1 Préparer le projet existant

Nous vérifions la présence du volume contenant le modèle déjà entraîné. Ce volume est ensuite monté en lecture seule dans le service API.

```
docker volume ls
```

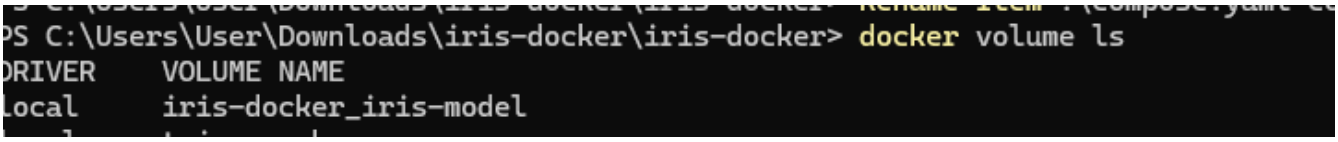


Figure 1: Volume existant du modèle Iris

Nous ouvrons requirements.txt et ajoutons redis==5.2.1 aux dépendances existantes pour permettre à l'API de communiquer avec Redis.

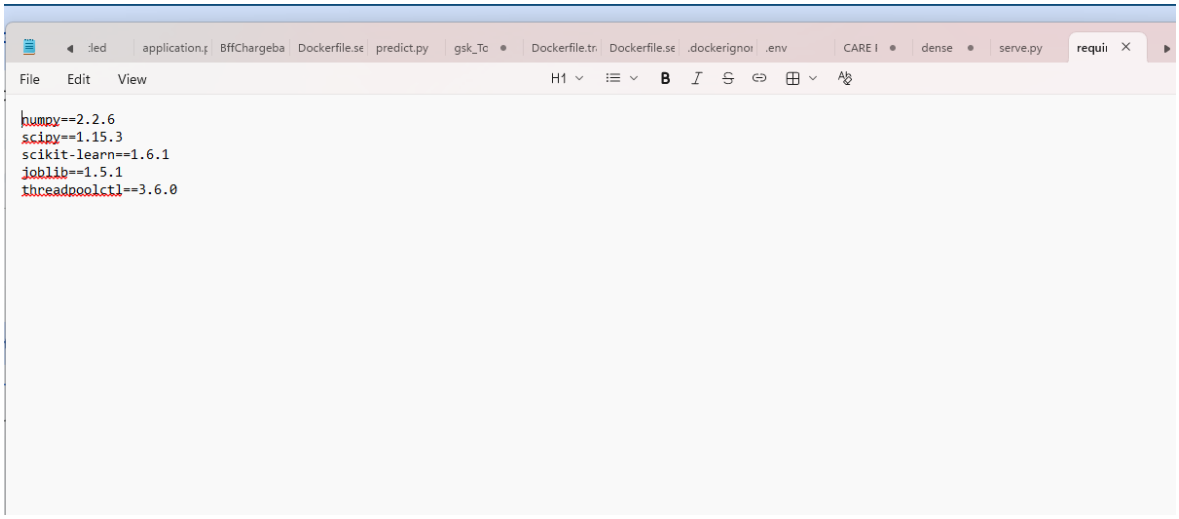


Figure 2: Fichier des dépendances avant l'ajout de Redis

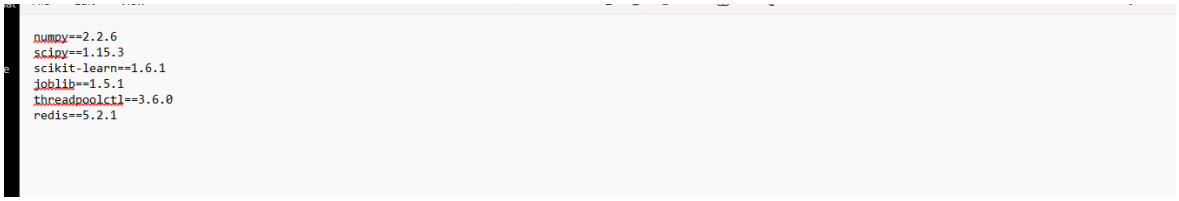


Figure 3: Ajout de redis==5.2.1 dans requirements.txt

2 Ajouter Redis à l'API

Nous modifions src/serve.py. Le modèle Iris reste chargé depuis le volume du TP précédent. Un client Redis est ajouté pour stocker le compteur, accessible sous la clé iris:predictions.

La connexion utilise la variable REDIS_HOST, avec base comme valeur par défaut. Ce nom correspond au service Redis déclaré dans Docker Compose.

```
import json
import math
import os

import joblib
import redis

from http.server import BaseHTTPRequestHandler, HTTPServer


bundle = joblib.load(
    os.environ.get("MODEL_PATH", "/models/iris.joblib")
)

database = redis.Redis(
    host=os.environ.get("REDIS_HOST", "base"),
    port=6379,
    decode_responses=True,
    socket_connect_timeout=3,
    socket_timeout=3,
)

COUNTER_KEY = "iris:predictions"


class Handler(BaseHTTPRequestHandler):
    def reply(self, status, payload):
        body = json.dumps(payload).encode("utf-8")

        self.send_response(status)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()

        self.wfile.write(body)

    def do_GET(self):
        if self.path == "/health":
            try:
                database.ping()
                self.reply(200, {"status": "ok"})
            except redis.RedisError:
                self.reply(503, {"error": "Redis indisponible"})
            return

        if self.path == "/stats":
            try:
                total = int(database.get(COUNTER_KEY) or 0)
                self.reply(200, {"predictions": total})
            except redis.RedisError:
                self.reply(503, {"error": "Redis indisponible"})
```

Figure 4: Extrait du code de l'API avec le client Redis et les routes de lecture

La route /stats lit la valeur du compteur dans Redis et renvoie 0 si la clé n'existe pas. La route /health vérifie également la disponibilité de Redis. Les réponses de /predict permettent ensuite de suivre l'augmentation du total des prédictions.

3 Configurer Docker Compose

Le fichier Compose définit deux services : api, construit à partir du Dockerfile d'inférence optimisé, et base, qui exécute Redis. L'API est accessible sur le port 8000 et utilise le nom base pour joindre Redis sur le réseau Compose.

File Edit View

```
services:
  api:
    build:
      context: .
      dockerfile: Dockerfile.serve.opt
    image: iris-api:redis
    ports:
      - "8000:8000"
    environment:
      MODEL_PATH: /models/iris.joblib
      REDIS_HOST: base
    volumes:
      - modele:/models:ro
    depends_on:
      base:
        condition: service_healthy

  base:
    image: redis:7.4-alpine
    command: ["redis-server", "--appendonly", "yes"]
    volumes:
      - redis-data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  modele:
    external: true
    name: iris-docker_iris-model

  redis-data:
```

Figure 5: Configuration des services API et Redis et des volumes

Le volume modele est déclaré externe et désigne iris-docker_iris-model. Le volume redis-data est monté dans /data. Redis active la persistance AOF avec –appendonly yes. La dépendance service_healthy attend que Redis réponde au contrôle redis-cli ping avant le démarrage de l'API.

```
docker compose config
docker compose config --services
```

```
  redis-data:
    name: iris-docker_redis-data
PS C:\Users\User\Downloads\iris-docker\iris-docker> docker compose config --services
base
api
```

Figure 6: Vérification des noms des services base et api

4 Démarrer et observer les services

```
docker compose up --build
```

Nous démarrons les services et observons les messages affichés. Compose construit l'image de l'API, récupère l'image Redis et crée les conteneurs ainsi que le volume de données Redis.

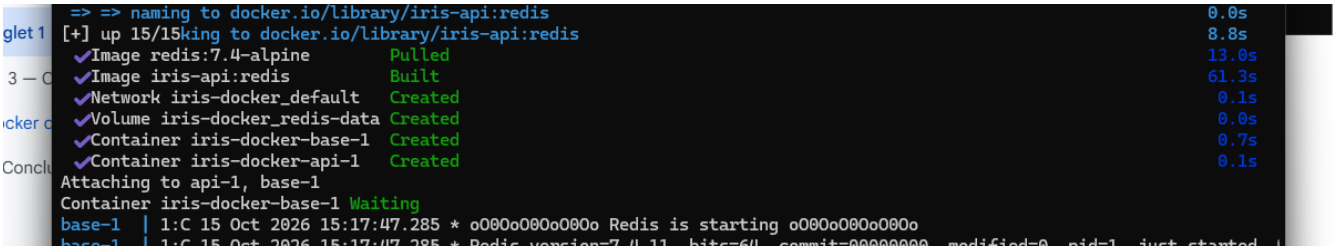


Figure 7: Création des ressources et démarrage des conteneurs

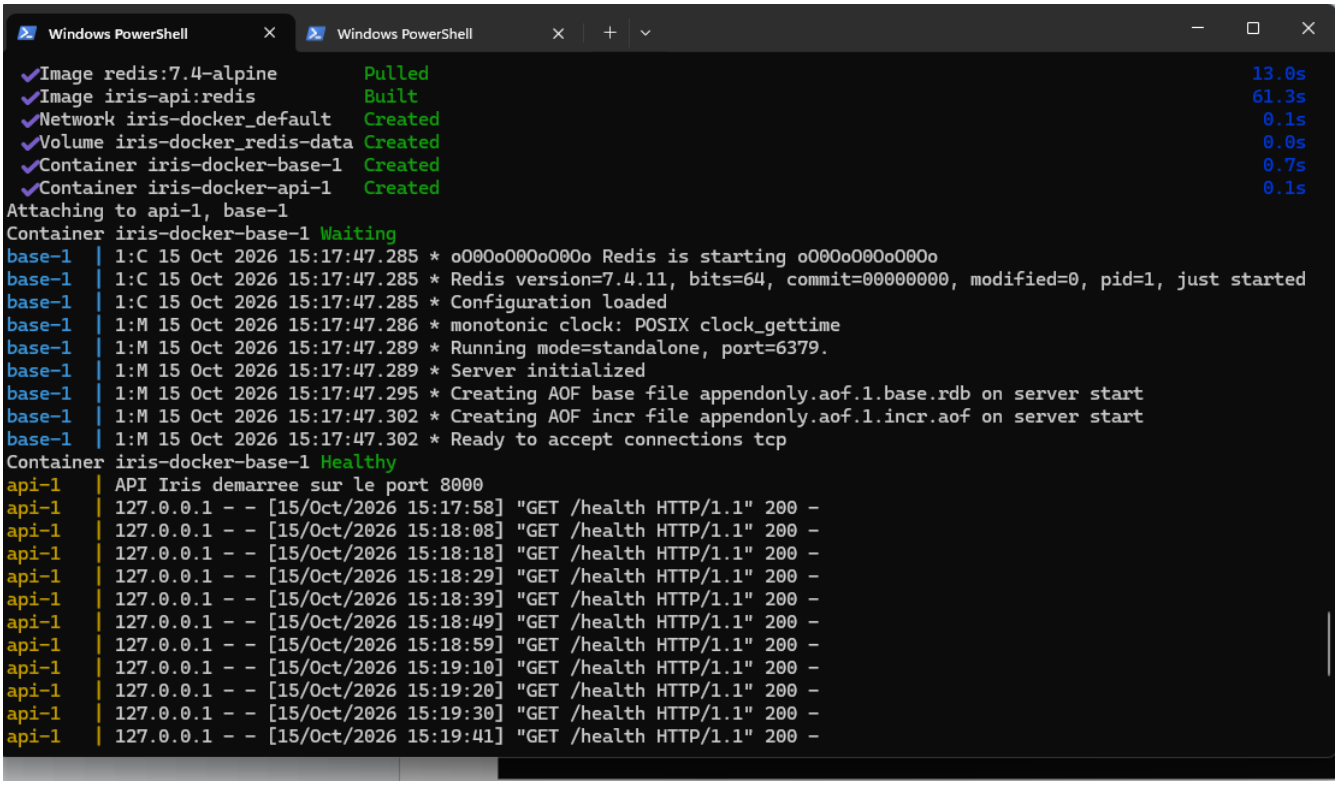


Figure 8: Journaux de Redis et de l'API après le démarrage

Les journaux indiquent que Redis est prêt à accepter les connexions. L'API annonce ensuite son démarrage sur le port 8000. Les appels périodiques à /health reçoivent une réponse HTTP 200.

5 Vérifier le compteur initial

Nous utilisons un second terminal PowerShell dans le dossier du projet pour consulter les services et interroger l'API.

```
docker compose ps
Invoke-RestMethod http://localhost:8000/stats
```

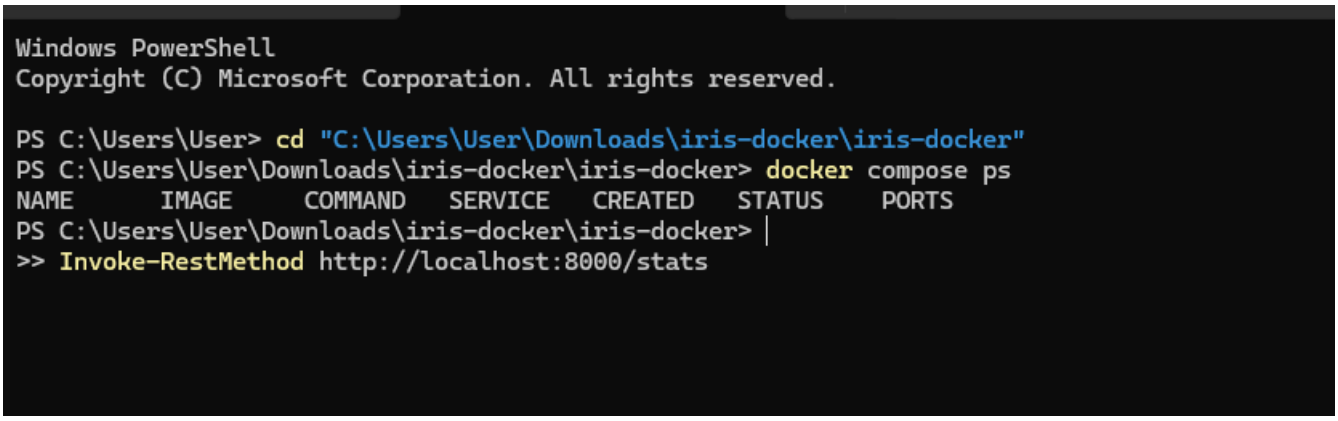


Figure 9: Commandes de consultation des services et du compteur

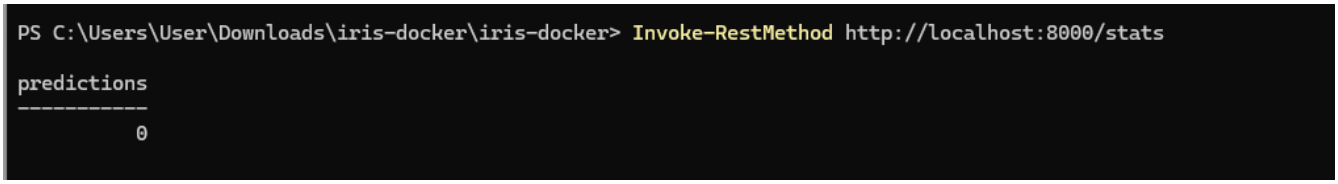


Figure 10: Lecture initiale du compteur à zéro

Avant l'envoi des requêtes de prédiction, /stats renvoie predictions = 0. Cette valeur constitue le point de départ du test d'incrémentation.

6 Envoyer cinq prédictions

Nous envoyons cinq requêtes à /predict à l'aide d'une boucle PowerShell. Chaque requête contient une fleur. Les réponses indiquent la classe prédite et le total des prédictions enregistrées.

```
PS C:\Users\User\Downloads\iris-docker\iris-docker> 1..5 | ForEach-Object {
>>     Invoke-RestMethod `
>>         -Uri http://localhost:8000/predict `
>>         -Method Post `
>>         -ContentType "application/json" `
>>         -Body $body
>> }

model         predictions total_predictions
-----         ----------- -----------------
random_forest {setosa}                  1
random_forest {setosa}                  2
random_forest {setosa}                  3
random_forest {setosa}                  4
random_forest {setosa}                  5
```

Figure 11: Cinq requêtes avec la classe setosa et un total croissant de 1 à 5

```
Invoke-RestMethod http://localhost:8000/stats
```

```
PS C:\Users\User\Downloads\iris-docker\iris-docker> Invoke-RestMethod http://localhost:8000/stats

predictions
-----------
          5
```

Figure 12: Compteur à cinq après les requêtes

Les cinq réponses retournent setosa avec le modèle random_forest. Le champ total_predictions augmente de 1 à 5. Une nouvelle lecture de /stats renvoie 5, ce qui confirme l'enregistrement du compteur dans Redis.

7 Tester la persistance du compteur

Nous arrêtons puis recréons les conteneurs avec docker compose down, suivi de docker compose up -d. La lecture de /stats renvoie toujours 5 : le volume Redis a été conservé.

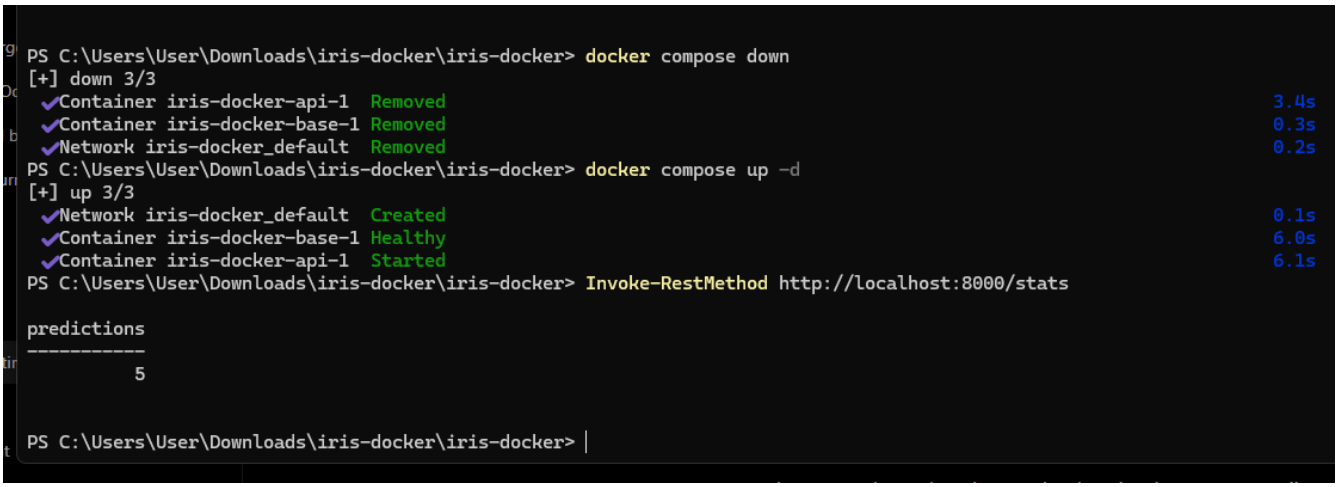


Figure 13: Compteur conservé après down puis up -d

Nous recommençons avec docker compose down -v. Cette commande supprime le volume redis-data. Au redémarrage, Redis utilise un volume vide et /stats renvoie 0. Le volume du modèle est conservé car il est déclaré externe.

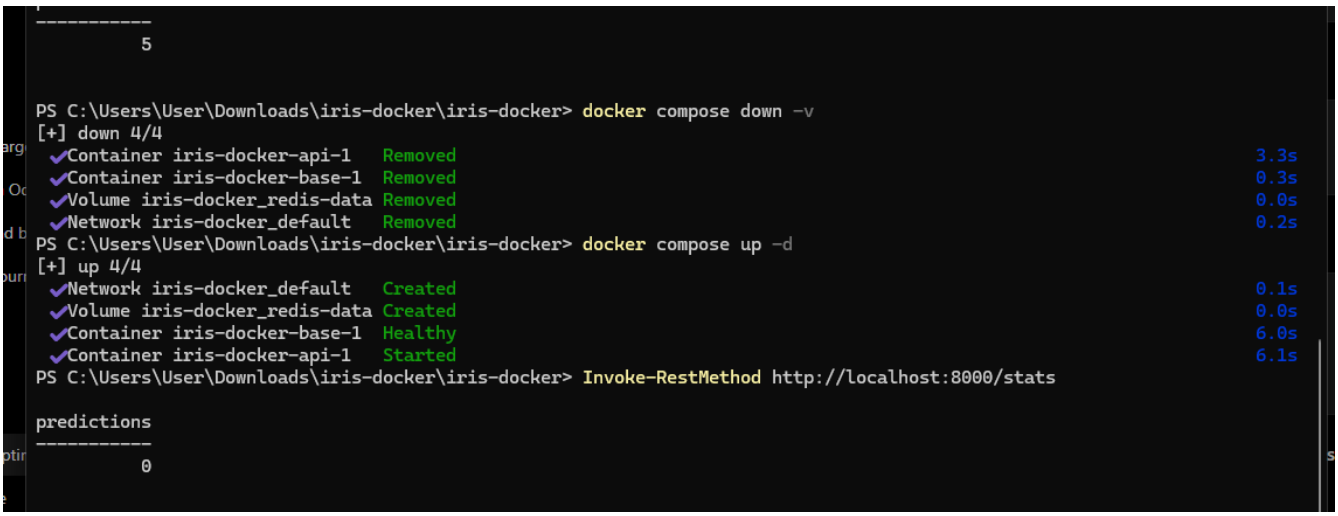


Figure 14: Suppression du volume Redis et remise à zéro du compteur

Bilan des tests

Test	Compteur observé	Interprétation
Premier démarrage	0	Aucune prédiction enregistrée
Cinq requêtes	5	Incrémentation du compteur
`down` puis `up -d`	5	Volume Redis conservé
`down -v` puis `up -d`	0	Volume Redis supprimé et recréé

Les tests montrent que la mémoire du compteur dépend du volume Redis. La recréation des conteneurs conserve les données tant que ce volume n'est pas supprimé.